

DOCUMENTO

Set e map

set-e-map.md

Sumário

Set e Map (set e map)	4
1. Por que existem set e map?	4
Parte 1: Conjunto (set)	5
2. A ideia	5
3. As operações	5
Percorrendo um set	6
Menor e maior	6
4. Procurando “o próximo”: lower_bound e upper_bound	6
5. set com outros tipos	7
Ordem decrescente	7
6. Quando o repetido importa: multiset	8
7. set ou vector?	8
8. Aplicação: o problema 2136 (Amigos do Habay)	8
O enunciado, em resumo	8
A dica	9
Parte 2: Mapa (map)	9
9. A ideia	9
10. As operações	9
O detalhe mais importante do map: m[chave] cria a chave	10
Percorrendo um map	11
lower_bound e upper_bound	11
11. Os usos clássicos	11
11.1 Contar frequências	11
11.2 Dar um número a cada nome	12
11.3 “Vetor” com índices enormes ou negativos	12
11.4 Agrupar	12
12. map ou vector?	12
13. Aplicação: o problema 1260 (Espécies de Madeira)	13

O enunciado, em resumo	13
A dica	13

Parte 3: As versões sem ordem (`unordered_set` e `unordered_map`) **13**

14. O que muda	13
15. Erros comuns	14

Set e Map (set e map)

Este material continua os anteriores de Introdução ao C++, `vector`, Busca Binária, Ordenação e Dois Ponteiros, e Pilha e Fila. Até aqui, para guardar dados, usamos estruturas que se importam com a **posição** (o `vector`) ou com a **ordem de chegada** (`stack` e `queue`). Agora vamos conhecer duas estruturas que se importam com **o que** está guardado:

1. O **conjunto** (`set`): guarda valores **sem repetição** e sempre **em ordem**;
2. O **mapa** (`map`): guarda pares **chave** → **valor**, como um dicionário, em que você procura pela chave e recebe o valor.

As duas respondem muito rápido a perguntas do tipo “esse valor já apareceu?” e “quantas vezes esse nome apareceu?”, que com `vector` exigiriam percorrer tudo a cada consulta.

1. Por que existem set e map?

Imagine que você lê 100.000 nomes e, para cada um, precisa saber se ele já apareceu antes. Com `vector`, a cada nome novo você teria que percorrer todos os anteriores:

```
bool jaApareceu = false;
for (const string& s : nomes)
    if (s == nome) { jaApareceu = true; break; }
```

cpp

Isso é um laço dentro de outro: 100.000 nomes vezes até 100.000 comparações. São bilhões de operações, e o programa estoura o tempo.

O `set` e o `map` foram feitos exatamente para isso. Eles guardam os dados de um jeito especial (uma árvore, por baixo dos panos) que permite **inserir, procurar e remover** qualquer elemento com poucas dezenas de comparações, mesmo com milhões de elementos guardados. Você não precisa saber como a árvore funciona; basta saber que:

- **inserir, procurar e remover** são rápidos;
- os elementos ficam **sempre ordenados**, de graça;
- não existe posição: **não dá para fazer** `s[0]` para pegar “o primeiro”.

NOTA

Eles vêm dos cabeçalhos `<set>` e `<map>`, mas com `#include <bits/stdc++.h>` já estão incluídos.

Parte 1: Conjunto (set)

2. A ideia

Um `set` é como um conjunto da matemática: uma coleção de elementos em que **cada valor aparece no máximo uma vez**. Se você inserir o 5 três vezes, ele fica lá uma vez só.

Além disso, o `set` mantém tudo **ordenado** automaticamente:

```
insert(7) insert(2) insert(9) insert(2) insert(5)

conteúdo: {2, 5, 7, 9}      (o segundo 2 foi ignorado)
```

3. As operações

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    set<int> s;

    s.insert(7);
    s.insert(2);
    s.insert(9);
    s.insert(2);           // repetido: não faz nada

    cout << s.size() << '\n';           // 3

    if (s.count(9)) cout << "tem 9\n"; // tem 9
    if (!s.count(4)) cout << "nao tem 4\n";

    s.erase(7);           // remove o 7

    for (int x : s) cout << x << ' '; // 2 9 (sempre em ordem)
    cout << '\n';
}
```

cpp

Função	O que faz
<code>s.insert(x)</code>	Inseri <code>x</code> ; se já existe, não faz nada
<code>s.erase(x)</code>	Remove <code>x</code> ; se não existe, não faz nada
<code>s.count(x)</code>	Devolve 1 se <code>x</code> está no conjunto, 0 se não
<code>s.contains(x)</code>	Devolve <code>true/false</code> (C++20; mesma coisa que <code>count</code> , mais legível)

Função	O que faz
<code>s.size()</code>	Quantos elementos há
<code>s.empty()</code>	true se está vazio
<code>s.clear()</code>	Esvazia
<code>*s.begin()</code>	O menor elemento
<code>*s.rbegin()</code>	O maior elemento

Percorrendo um `set`

O `for` de intervalo funciona e visita os elementos **em ordem crescente**:

```
for (int x : s) cout << x << ' ';
```

cpp

Isso significa que “inserir tudo em um `set` e percorrer” é um jeito de **remover repetidos e ordenar** em um passo só.

Menor e maior

`s.begin()` aponta para o menor elemento e `s.rbegin()` para o maior. O `*` na frente serve para “pegar o valor apontado”:

```
cout << *s.begin();    // menor  
cout << *s.rbegin();   // maior
```

cpp

CUIDADO

Assim como `top()` na pilha, `*s.begin()` em um `set` **vazio** faz o programa se comportar de forma imprevisível. Confira `empty()` antes.

4. Procurando “o próximo”: `lower_bound` e `upper_bound`

Você já viu essas funções no material de Busca Binária, usadas em vetores ordenados. O `set` tem as **suas próprias versões**, que funcionam do mesmo jeito, mas devem ser chamadas como `s.lower_bound(x)` (e não `lower_bound(s.begin(), s.end(), x)`, que existe mas é lento em `set`).

- `s.lower_bound(x)`: aponta para o **primeiro elemento maior ou igual** a `x`;
- `s.upper_bound(x)`: aponta para o **primeiro elemento maior** que `x`.

Se não existe tal elemento, elas devolvem `s.end()`, que é um “aponta para nada”. Por isso sempre teste antes de usar o `*`:

```
set<int> s = {2, 5, 7, 9};

auto it = s.lower_bound(6);    // aponta para o 7
if (it != s.end()) cout << *it;    // 7

it = s.lower_bound(10);        // nada é >= 10
if (it == s.end()) cout << "nao existe";
```

cpp

Isso resolve perguntas como “qual o menor valor guardado que é pelo menos `x`?” sem percorrer nada.

Para o **maior elemento menor que** `x`, ande um passo para trás a partir do `lower_bound`, tomando cuidado com o começo:

```
auto it = s.lower_bound(x);
if (it != s.begin()) {
    --it;    // agora aponta para o maior elemento < x
    cout << *it;
}
```

cpp

5. set com outros tipos

Funciona com `string`, `pair`, `char`, `long` `long`... qualquer tipo que tenha o operador `<`, porque é ele que define a ordem (a mesma regra que você viu no material de ordenação):

```
set<string> nomes;
nomes.insert("Maria");
nomes.insert("Ana");
nomes.insert("Maria");
for (const string& n : nomes) cout << n << '\n';    // Ana, Maria
```

cpp

Um `set<pair<int, int>>` ordena pelo `first` e desempata pelo `second`, também como no `sort`.

Ordem decrescente

```
set<int, greater<int>> s;    // percorrer dá do maior para o menor
```

cpp

6. Quando o repetido importa: `multiset`

Se você **precisa** guardar valores repetidos, mas ainda quer tudo ordenado e com busca rápida, use `multiset`. Funciona igual ao `set`, com uma diferença importante:

```
multiset<int> m;
m.insert(5);
m.insert(5);
m.insert(3);

cout << m.count(5);      // 2  (quantas vezes o 5 aparece)

m.erase(5);              // CUIDADO: remove TODOS os 5
```

cpp

Para remover **apenas uma cópia**, remova pelo “endereço” que o `find` devolve:

```
m.erase(m.find(5));      // remove só um 5
```

cpp

(Se o valor não existir, `find` devolve `m.end()`, e apagar isso é imprevisível. Confira antes com `count`.)

7. `set` ou `vector`?

Preciso de...	Use
Saber rápido se um valor existe	<code>set</code>
Sem repetidos, sempre ordenado	<code>set</code>
Inserir e remover no meio, muitas vezes	<code>set</code>
Acessar pela posição (<code>v[i]</code>)	<code>vector</code>
Percorrer tudo, ou ordenar uma vez só no fim	<code>vector</code> + <code>sort</code>
Guardar repetidos	<code>vector</code> ou <code>multiset</code>

Uma regra prática: se todos os dados chegam de uma vez e você só precisa ordenar e olhar, `vector` + `sort` é mais simples e mais rápido. Se as inserções, remoções e consultas ficam **misturadas** ao longo do programa, o `set` brilha.

8. Aplicação: o problema 2136 (*Amigos do Habay*)

O enunciado, em resumo

Cada linha da entrada traz um nome e a opção YES ou NO, até aparecer a palavra FIM. Quem escolheu YES pode ter se inscrito **várias vezes**; quem escolheu NO, só uma. Você deve imprimir todos os nomes com YES em ordem alfabética (cada nome uma vez só), depois todos com NO em ordem alfabética, e por fim o “Amigo do Habay”: o nome com **mais letras** entre os YES, e em caso de empate o que se inscreveu **primeiro**.

A dica

Leia de novo o que precisa ser impresso na lista: nomes **sem repetição** e **em ordem alfabética**. Essas são exatamente as duas coisas que o set faz sozinho, sem você escrever nem um sort nem uma verificação de “já apareceu”. Pense em quantos conjuntos você precisa.

Já o “Amigo do Habay” não depende de ordem alfabética, e sim da ordem em que os nomes chegam. Isso não tem nada a ver com o set: dá para decidir enquanto você lê a entrada, com uma variável só. Cuidado com o desempate.

Parte 2: Mapa (map)

9. A ideia

Um map é um **dicionário**: você guarda pares **chave** → **valor** e consulta pelo valor da chave.

```
"Ana"    → 3  
"Bruno"  → 1  
"Carla"  → 7
```

É como um vector, só que o índice não precisa ser um número de 0 a n-1: pode ser uma string, um número enorme, um pair, qualquer coisa com <. Cada chave aparece **uma vez só** e as chaves ficam **sempre ordenadas**, como no set.

10. As operações

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    map<string, int> idade;

    idade["Ana"] = 17;           // cria a chave "Ana" com valor 17
    idade["Bruno"] = 16;
    idade["Ana"] = 18;          // sobrescreve o valor de "Ana"

    cout << idade["Ana"] << '\n';      // 18
    cout << idade.size() << '\n';      // 2

    if (idade.count("Carla")) cout << "tem Carla\n";
    else cout << "nao tem Carla\n";

    idade.erase("Bruno");

    for (auto [nome, anos] : idade)      // em ordem alfabética da chave
        cout << nome << " tem " << anos << " anos\n";
}
```

cpp

Função	O que faz
<code>m[chave]</code>	Acessa o valor da chave; se ela não existe, cria com valor “zero”
<code>m[chave] = v</code>	Cria ou sobrescreve
<code>m.count(chave)</code>	1 se a chave existe, 0 se não
<code>m.contains(chave)</code>	true/false (C++20)
<code>m.erase(chave)</code>	Remove a chave e o valor
<code>m.size(), m.empty(), m.clear()</code>	Como sempre
<code>m.begin()->first</code>	A menor chave
<code>m.rbegin()->first</code>	A maior chave

O detalhe mais importante do `map`: `m[chave]` **cria a chave**

Quando você faz `m["Zé"]` e “Zé” **não existe**, o `map` **cria** a chave “Zé” com o valor padrão do tipo (0 para números, "" para strings, false para bool). Isso é ótimo para contar:

```
map<string, int> cont;
cont["Ana"]++;      // "Ana" não existia: vira 0, depois 1
cont["Ana"]++;      // 2
```

cpp

Mas é uma armadilha se você só queria **consultar**:

```
if (m["Carla"] > 0) ... // se "Carla" não existia, agora existe com 0!
```

cpp

Depois disso, `m.size()` aumentou e um `for` pelo `map` vai imprimir “Carla 0”. Para **só perguntar** se uma chave existe, use `count` ou `contains`.

Percorrendo um `map`

Cada elemento do `map` é um `pair`: o `.first` é a chave e o `.second` é o valor. O jeito mais legível é dar nome aos dois na hora:

```
for (auto [chave, valor] : m)
    cout << chave << ": " << valor << '\n';
```

cpp

Os pares saem em **ordem crescente de chave**. Se quiser alterar o valor dentro do laço, use `auto&`:

```
for (auto& [chave, valor] : m) valor *= 2;
```

cpp

`lower_bound` e `upper_bound`

Existem no `map` também, com o mesmo significado do `set`, só que aplicados às chaves. Devolvem um “apontador” para o par; use `->first` e `->second`:

```
auto it = m.lower_bound("B"); // primeira chave >= "B"
if (it != m.end()) cout << it->first << ' ' << it->second;
```

cpp

11. Os usos clássicos

11.1 Contar frequências

O uso número um. “Quantas vezes cada palavra aparece?”, “qual número aparece mais?”:

```
map<string, int> freq;
string palavra;
while (cin >> palavra) freq[palavra]++;

for (auto [p, q] : freq) cout << p << ' ' << q << '\n'; // já sai em ordem
```

11.2 Dar um número a cada nome

Muitos problemas dão nomes (de cidades, pessoas, times) e você quer trabalhar com índices de 0 a n-1 em um `vector`. O `map` faz a tradução:

```
map<string, int> id;
vector<string> nomeDoId;

int pegaId (const string& nome) {
    if (!id.count(nome)) {           // primeira vez que aparece
        id[nome] = nomeDoId.size(); // recebe o próximo número livre
        nomeDoId.push_back(nome);
    }
    return id[nome];
}
```

11.3 “Vetor” com índices enormes ou negativos

Se o problema fala em posições que vão até 1.000.000.000, um `vector` desse tamanho não cabe na memória. Mas se só **algumas** dessas posições são usadas, um `map<long long, int>` guarda apenas as que aparecem.

11.4 Agrupar

`map<int, vector<string>>`: para cada chave, uma lista. Por exemplo, “todos os alunos de cada turma”:

```
map<int, vector<string>> turma;
turma[2].push_back("Ana");
turma[2].push_back("Bia");
turma[1].push_back("Caio");

for (auto [t, alunos] : turma) {           // turma 1, depois turma 2
    cout << "Turma " << t << ":";
    for (const string& a : alunos) cout << ' ' << a;
    cout << '\n';
}
```

12. `map` ou `vector`?

Se as chaves são números pequenos e “densos” (de 0 a n, digamos até uns milhões), um `vector` indexado por elas é mais simples e muito mais rápido:

```
vector<int> cont(1000001, 0); // conta números de 0 a 1.000.000
cont[x]++;
```

cpp

Use `map` quando a chave **não é um número pequeno**: strings, pares, números grandes ou negativos, ou quando você não sabe de antemão o intervalo das chaves.

13. Aplicação: o problema 1260 (*Espécies de Madeira*)

O enunciado, em resumo

Para cada caso de teste, a entrada traz uma lista de nomes de árvores, um por linha (nomes podem ter espaços), até uma linha em branco. Você deve imprimir cada espécie **em ordem alfabética**, seguida do **percentual** que ela representa do total de árvores daquele caso, com 4 casas decimais. Pode haver até 1.000.000 de árvores e 10.000 espécies por caso.

A dica

“Quantas vezes cada nome apareceu” com um nome que é uma `string`: essa é a frase que descreve o `map` de frequências da seção 11.1. E a saída em ordem alfabética vem de graça ao percorrer o `map`.

Para o percentual você vai precisar do total de árvores do caso e de uma divisão com decimais. Lembre-se de que dividir dois `int` dá um `int`; force uma das partes a ser `double`. Para imprimir com 4 casas, use `fixed << setprecision(4)`.

Dois cuidados de leitura: os nomes têm **espaços**, então use `getline` (e o `cin.ignore()` que você conheceu no material de pilha e fila); e há uma linha em branco entre os casos, que serve de sinal de “acabou este caso”. Não esqueça de limpar o `map` e o total antes de começar o próximo.

Parte 3: As versões sem ordem (`unordered_set` e `unordered_map`)

14. O que muda

Existem duas variantes, `unordered_set` e `unordered_map`, com **as mesmas funções** (`insert`, `erase`, `count`, `m[chave]`...), mas que guardam os dados de outro jeito (uma *tabela hash*, em vez de árvore). Consequências:

- na média, inserir e procurar é **ainda mais rápido**;
- os elementos **não ficam em ordem nenhuma**; percorrer dá uma ordem “aleatória”;
- **não têm** `lower_bound/upper_bound`, nem `begin()` como “menor”;
- em casos raros (entradas feitas de propósito para atrapalhar), podem ficar muito lentas.

Regra prática para prova: comece com `set/map`. Se o programa está estourando o tempo, você só usa `insert/count/m[chave]` e não precisa de ordem, troque para a versão `unordered_`. Só não use `unordered_` com `pair` como chave sem escrever uma função de hash própria: o C++ não traz uma pronta para `pair`.

15. Erros comuns

No `set`:

- Tentar `s[0]` ou `s[i]`. Não existe posição. Para o menor, `*s.begin()`; para o maior, `*s.rbegin()`; para percorrer, `for (auto x : s)`.
- Usar `lower_bound(s.begin(), s.end(), x)` em vez de `s.lower_bound(x)`. Compila, mas fica lento.
- Usar o `*` em `s.end()`, ou em um `lower_bound` sem verificar se deu `s.end()`.
- Esperar que o `set` guarde repetidos e contar `s.size()` como “total de elementos lidos”.
- No `multiset`, chamar `m.erase(valor)` querendo remover só uma cópia.

No `map`:

- Consultar com `m[chave]` e, sem querer, **criar** a chave. Para perguntar se existe, use `count` ou `contains`.
- Percorrer com `for (auto p : m)` e esquecer que `p` é um `pair`: a chave é `p.first` e o valor é `p.second`. Ou use `for (auto [k, v] : m)`.
- Alterar valores dentro do `for` sem `&` e ficar alterando uma cópia.
- Usar `map` para chaves que são números de 0 a `n` pequeno, quando um `vector` seria mais simples e rápido.

Nos dois:

- Modificar o `set/map` (inserir ou apagar) **enquanto** percorre com um `for` de intervalo. O laço se perde. Se precisar, guarde o que vai apagar em um `vector` e apague depois.
- Declarar a estrutura fora do laço de casos de teste e esquecer o `clear()` entre um caso e outro.
- Usar `unordered_` e depois precisar da ordem, ou usar `pair` como chave nele sem hash próprio.